Module-IX
Lecture-II and III
D-Algorithm

1. Introduction

In the last module we have discussed about a new algebra called Routh's algebra, which is suitable for ATPG using the "sensitize-propagate-justify" approach. In this chapter we will discuss Roth's D-Algorithm [1] in depth, which is the first formal algorithm for ATPG. We will first start with the basic definitions and procedures required in D-algorithm. Following that we will explain D-algorithm with examples. Finally, we will see in brief, some issues in D-algorithm, and how they were addressed in some advanced algorithms.

2. Definitions and procedures

Now we will see some definitions and procedures required in D-algorithm.

Definition 1: Singular cover

Singular cover of a logic gate is the minimal set of input signal assignments needed to represent essential prime implicants in the Karnaugh map of the logic gate, both for the case 0 and case 1. Table 1 shows singular cover for 2 input AND and 2 input OR gate.

Table 1. Singular cover for 2 input AND and 2 input OR gate

AND	Inputs A	B	Output		OR	Inputs A	B	Output
	0	X	0			1	X	1
	X	0	0			X	1	1
	1	1	1			0	0	0

From the table it may be noted that for 0 at the output of the AND gate only one input needs to be specified and the other is X. So we have 2 choices (0X and X0) for the singular cover of a 2-input AND gate for output =0. But for output =1 there is only one choice. Similarly, for the OR gate we have 2 choices (1X and X1) for the singular cover for output =1; for output=0 we have a single choice. In a similar way, singular cover for any gate can be found.

Definition 2: D-frontier

The D-frontier comprises gates whose output value is X and at least one of its input is D or $\bar{D}$. This implies that, any gate in D-frontier can be used for fault propagation. This is explained by an example given in Figure 1.

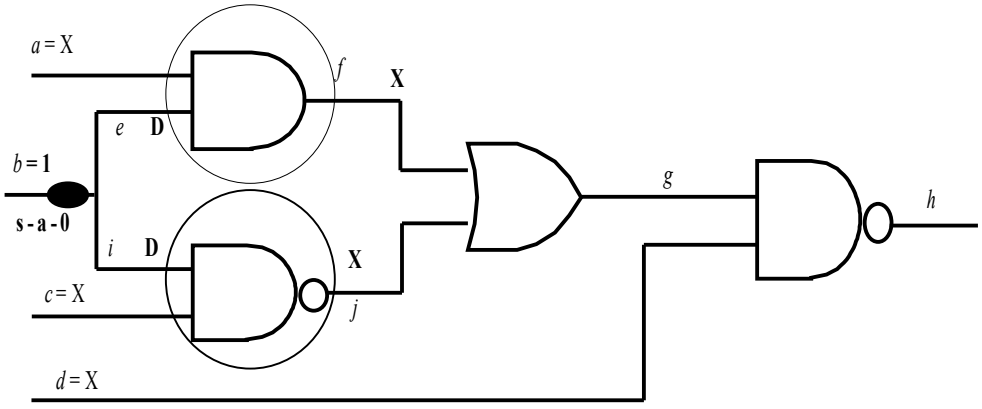


Figure 1. D-frontier-the encircled gates

In Figure 1, let us consider a s-a-0 fault at input line *b*. The only way to sensitize the fault is to make *b*=1. Automatically, nets *e* and *i* are assigned D. All other nets are assumed to have the value X. Now, D-frontier comprises gates which are encircled. It may be noted that in these gates one input is D and the other is X. The basic idea is, X at the inputs can be appropriately selected so that D or $\bar{D}$ can be propagated to the output of the gates. In other words, faults can be propagated only through gates in the D-frontier. Once the fault is propagated (i.e., the output of the gate has D or $\bar{D}$), the gate is deleted from the D-frontier list.

Definition 3: Unique D-Drive

If there is only one gate in D-frontier, then fault effect has to be propagated through that gate. This situation is called unique D-drive.

Definition 4: J-frontier

The J-frontier comprises gates whose output value is known (0 or 1) but its inputs are not yet computed (or not yet implied by its inputs). In other words, output of a gate in the J-frontier is known, but inputs are not yet computed. This implies that, any gate in J-frontier can be used for justification. This is explained by an example given in Figure 2.

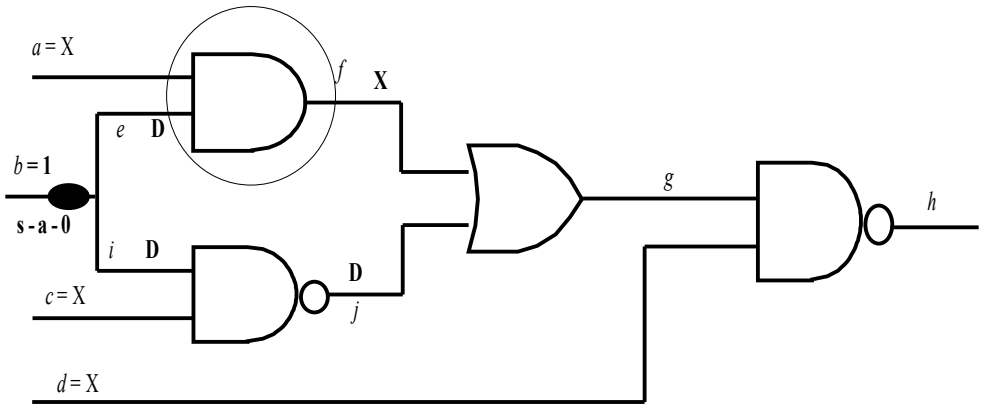


Figure 2. J-frontier--the encircled gates

We consider the same circuit as in Figure 1. Assume that it was decided that fault effect D be propagated via *j*. So, *j*=D by forward implication. Also, *f*=0 for propagating D to *g*. Now, the encircled gate in Figure 2 is in the J-frontier, because the output is known as 0 and its inputs are X, which can be decided in *justification* step. The basic idea is, X at the inputs can be appropriately selected so that output is justified. In other words, during justification gates in J-frontier can only be considered. Once the inputs are justified, the gate is deleted from the J-frontier list.

Procedure 1: Implication

The implication procedure comprises 3 steps

- Compute all the values of the signals that can be determined uniquely from already given signal values (of some nets). If many choices are available, for example as in singular cover, any one of them can be considered.
- Maintain J-frontier and D-frontiers
- Check for consistency and stop in case of inconsistency (i.e., contradictory signal values are implied by the implication procedure).

Basically, implication procedure is similar to a simulation process where values of all nets (and finally primary outputs) are determined starting from primary inputs. However there are some differences between simulation and implication as discussed in Table 2.

Table 2: Simulation and Implication

Simulation	**Implication**
All the signal values are determined uniquely	All signals may not be determined uniquely
Value assignment moves from inputs to outputs of a circuit	Signal assignments propagate both towards primary inputs and primary outputs. For example, to test a s-a-1 fault at net *l* say, we need to have implication in two directions (i) backwards, to primary inputs to make *l*=0 and (ii) forward, to primary outputs to propagate D.
There is no inconsistency	Inconsistency may arise, when for a given net *l* (which is not the fault location) different signal values (0 or 1) need to be assigned.

Figure 3 explains the forward implication procedure.

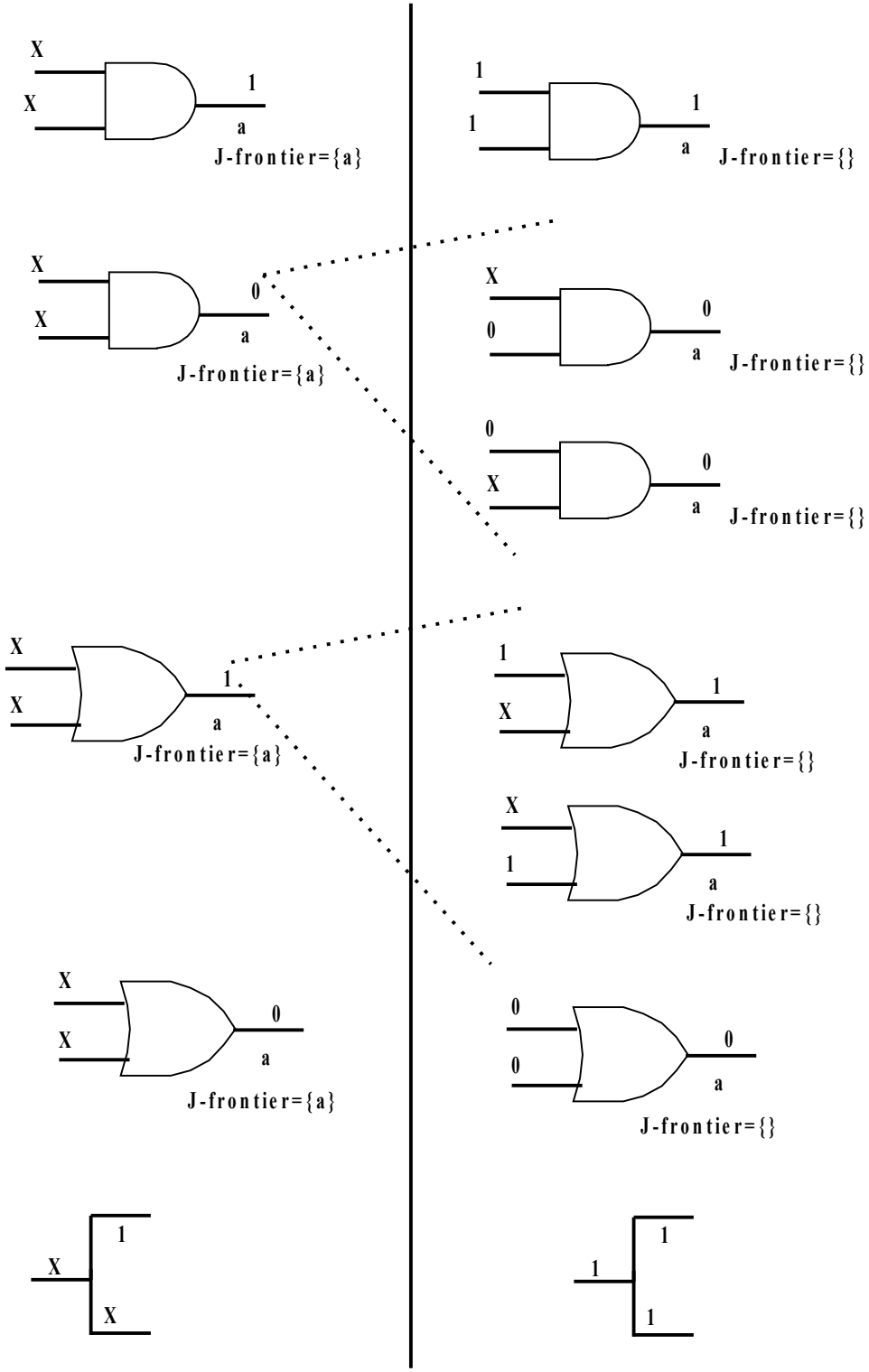


Figure 3. Backward implication and J-frontier

In Figure 3, gates at left side represent the case before backward implication and the ones at the right side illustrate the situation after implication The first gate in the sequence (from top) has inputs as X and output as 1; so J-frontier is a. Now by backward implication (i.e., singular cover) both inputs are to be 1. The corresponding gate in right side shows that inputs are decided by backward implication and so gate (a) is removed from J-frontier. In Figure 3, the second gate has output of 0 and so by singular cover we have two options for assigning values to the inputs; the two options are shown in the right side. It may be noted that we could have also kept another option where both the inputs are kept 1. However, this is avoided because if both inputs are fixed then flexibility gets reduced and chances of inconsistency increases. This is explained by an example given in Figure 4. In the figure it may be noted that in Step-2 we have three options (at the input) to obtain D at the output of the AND gate--00,01 and 10. If we choose 10 then we will have inconsistency in Step-4, because to propagate D through the OR gate the input not carrying D is to be 0. To avoid such inconsistencies we maintain as many X as possible to have flexibility for successful forward and backward implications.

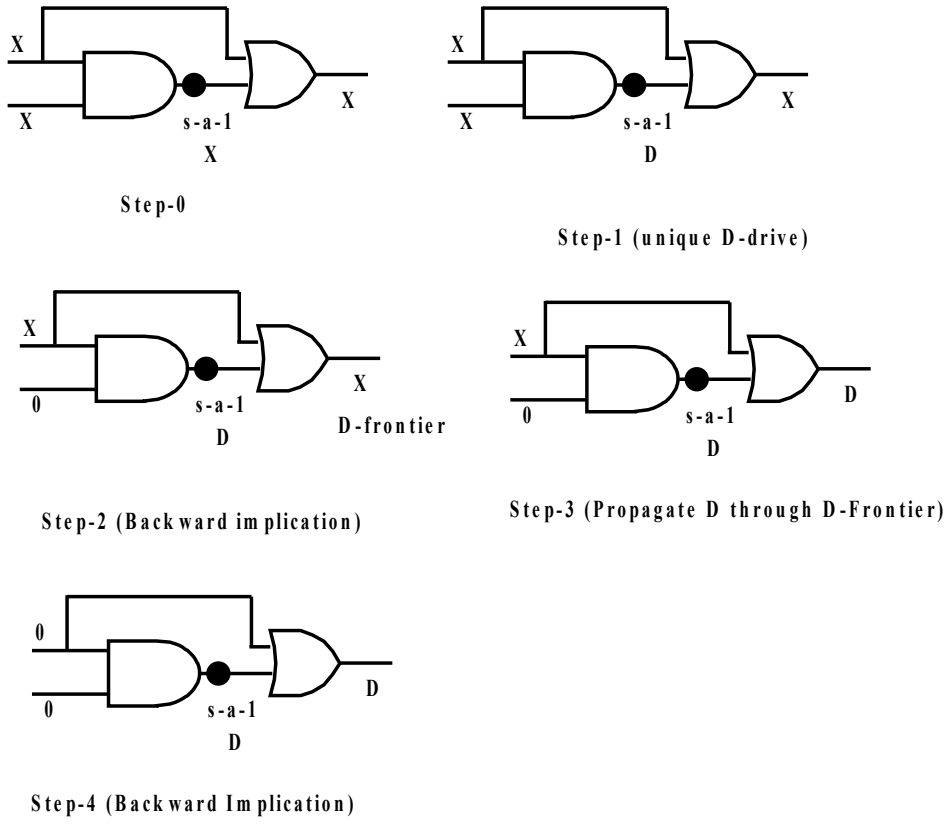


Figure 4. Propagating D using backward and forward implication

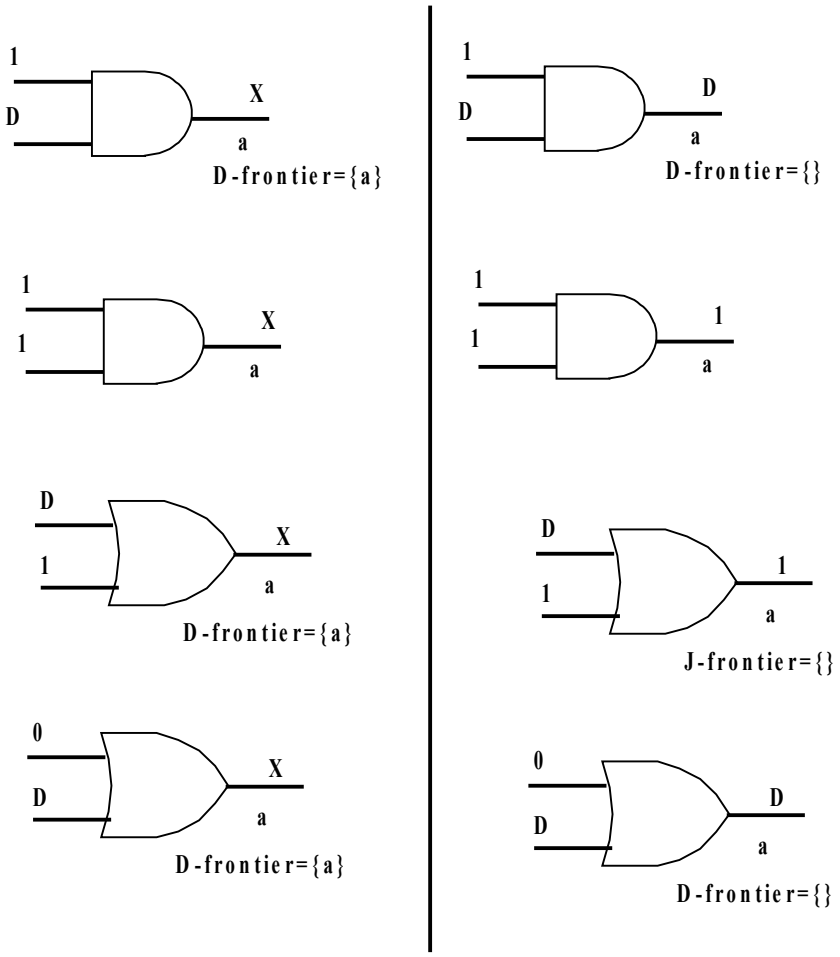


Figure 5. Forward Implication and D-frontier

Figure 5 illustrates examples of forward implications. The first gate in the figure shows how D is propagated to output and then D-frontier removes the gate from the list. Other cases can be explained in a similar manner.

Procedure 5: X-path

An X-path is a path of consecutive nets in a circuit all of whose values are X. Let A be a gate in a D-frontier. The faults on the inputs of A can be propagated to a primary output O only if there is an X-path from A to O. In Figure 6 there is only one X-path from *a* to output—*a-c-d.*

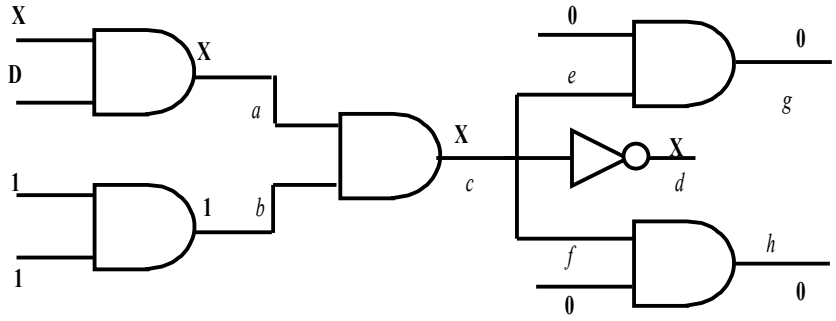


Figure 6. Example of an X-path

3. **D-Algorithm**

In this section we will first present the steps involved in D-algorithm. Following that we will elaborate the algorithm using an example.

a. **D-Algorithm**

Input: Circuit netlist and one stuck at fault with its location.

Output: Primary input values and expected values at the primary outputs

Step-1: Initialize all the signal values of the circuit to X.

Step-2: Sensitize the fault location, i.e., if s-a-0 is the fault, apply D in the fault location, else if s-a-1 is the fault, apply $\overline{D}$ in the fault location.

Step 3: Determine/update gates in D-frontier and J-frontier, due to signal changes from X to 0,1, D or $\overline{D}$. If there is no gate in D-frontier and D / $\overline{D}$ has not reached any primary output, backtrack().

Step-4: If D / $\overline{D}$ has not reached any primary output, propagate D / $\overline{D}$ through one of the gates in the D-frontier.

Step-5: Use forward and backward implications to determine as many signal values as possible. backtrack() in case of an inconsistency.

Step-6: If D / $\overline{D}$ has reached any primary output and the primary inputs have received values by backward implications, stop; the values of the primary inputs comprise the test pattern. Else, go to Step-3.

The module backtrack(), works as follows.

Step-1: Determine the last choice (of alternatives) taken in selection of gates in D-frontier or signal values in forward/backward implications, where another new alternative is available. Take any one of the other alternative. If no choice is left, ***report non testable fault*** and ***return***.

Step-2: Undo all steps (i.e., propagation of D / $\overline{D}$ and forward/backward implications), from the current signal values to point where another alternative is attempted. **Return**.

b. **An example**

The D-algorithm discussed above will now be illustrated using an example. Consider a simple circuit as shown in Figure 7(a). The circuit has a s-a-0 fault at the output of gate-1. Also as mentioned in Step-1 of the algorithm all nets have been initialized to X.

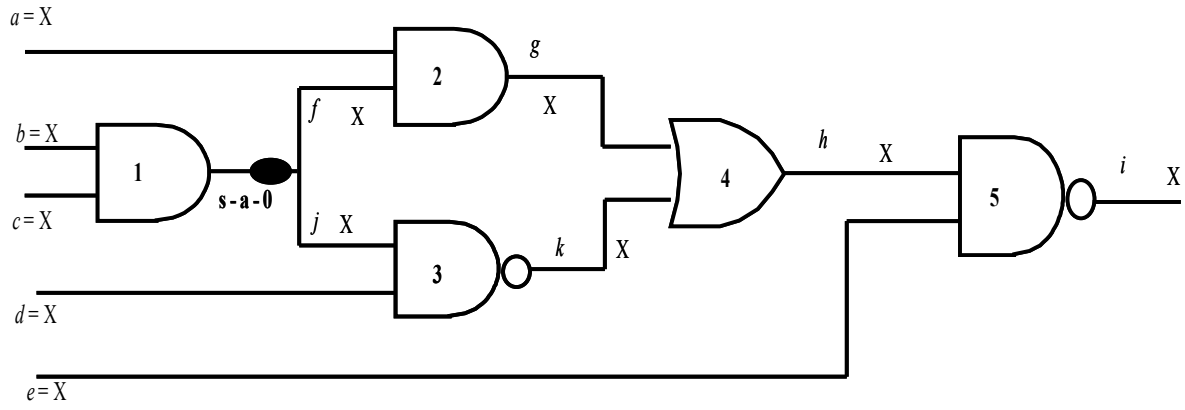


Figure 7(a) Example of simple digital circuit with a s-a-0 fault

To sensitize the s-a-0 fault, 1 has to be applied in the corresponding net, thereby making signal of the net as D. Now J- frontier and D-frontier needs to be computed. As the output of gate 1 is known (as D) and its inputs are X, J-frontier is gate-1. Since one of the inputs of gate-2 and gate-3 are D, and the outputs are X, D-frontier is gate-2 and gate-3. This step is illustrated in Figure 7(b).

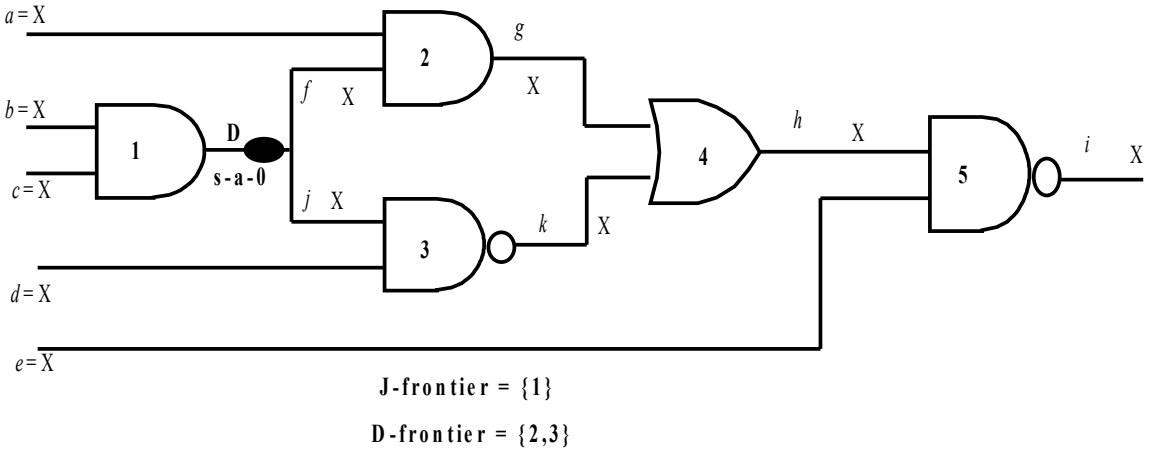


Figure 7(b): Sensitizing the fault location and computation of J- and D-frontier

To justify D at output of gate-1, if we use backward implication we get *a*=1 and *b*=1. In other words, backward implication in gate-1 (J-frontier) generates *a*=1 and *b*=1. This step is illustrated in Figure 7(c). Now, J-frontier has no gates.

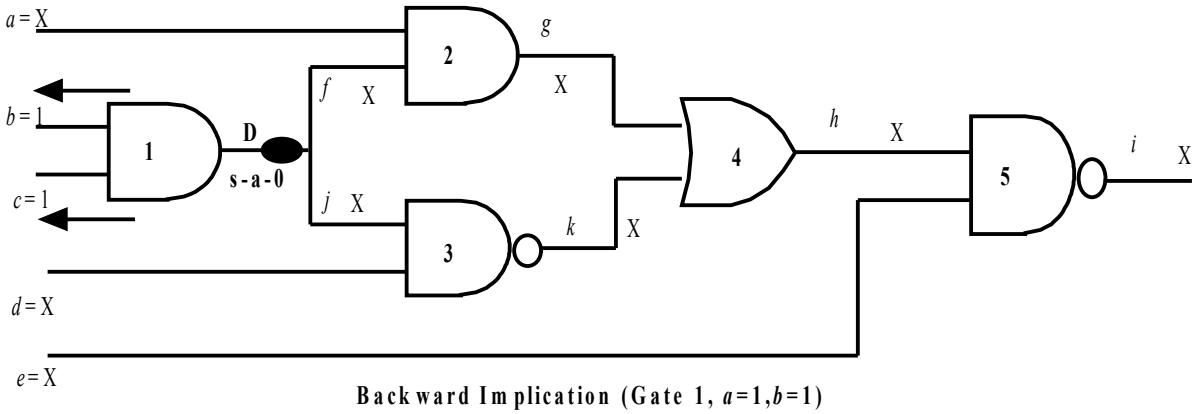


Figure 7(c): Backward implication at gate-1

If we observe Figure 7(b), we see that there are 2 gates in D-frontier, namely 2 and 3. So we can choose one for fault propagation. Let us first try gate 2. So net *f* =D and *j*=X. This is illustrated in Figure 7(d). Now, gate 3 is removed from D-frontier and only gate 2 remains.

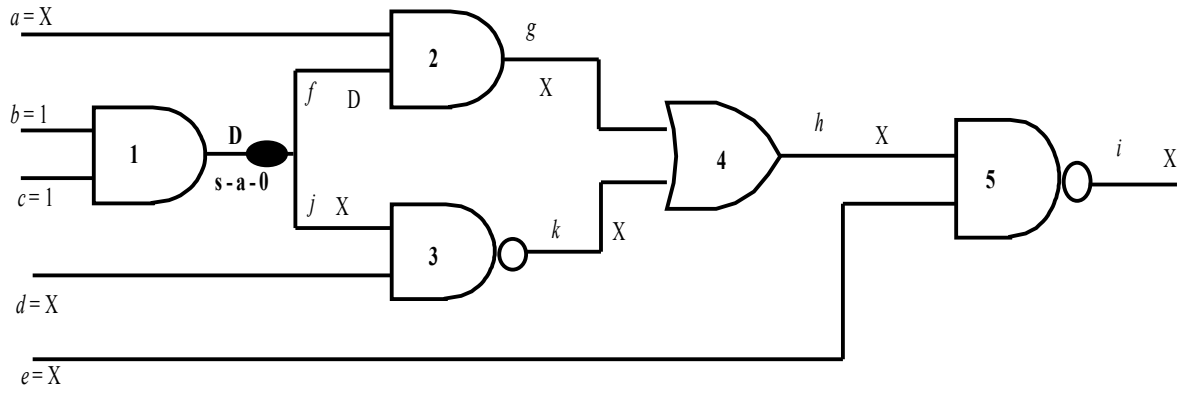
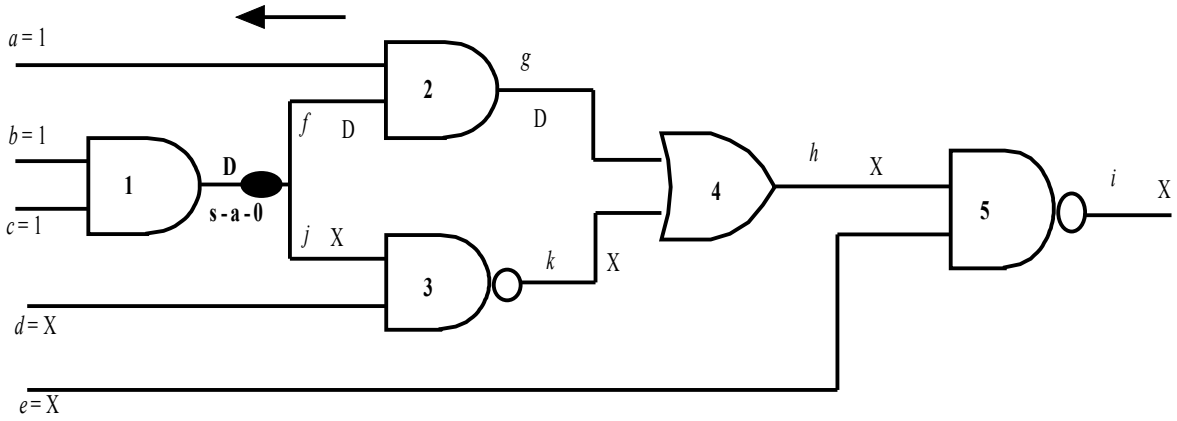


Figure 7(d): Selection of gate-2 as D-frontier

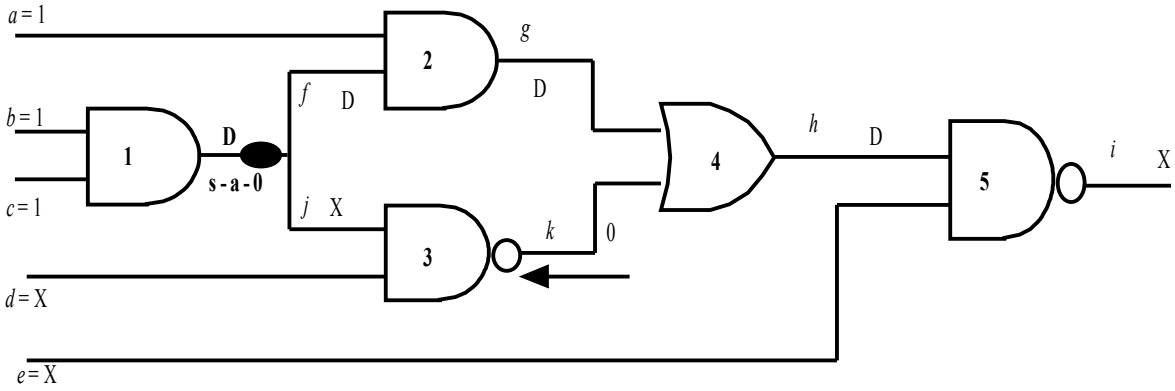
In the next step, we propagate D though gate-2 (only choice in the D-frontier). This makes net *g*=D by forward implication and *a*=1 by backward implication. The step is illustrated in Figure 7(e). After propagation of D though gate-2, the new D-frontier comprises gate-4.



Propagate D through D-frontier and Backward Implication
Modified D-frontier ={4}

Figure 7(e): Propagation of D and implications

In the next step, we propagate D through gate-4 (only choice in the D-frontier). This makes net *h*=D by forward implication. So D-frontier comprises gate-5. Now by backward implication net *k*=0. This creates a new J-frontier comprising gate-3; output of gate-3 is 0 and both of its inputs are X. The step is illustrated in Figure 7(f).



Propagate D through D-frontier and Backward Implication
Modified D-frontier ={5}
J-frontier={3}

Figure 7(f): Propagation of D, implications and J-frontier

In the next step, backward implication of the gate at J-frontier (gate-3) implies net *j*=1 and primary input *d*=1. This is illustrated in Figure 7(g).

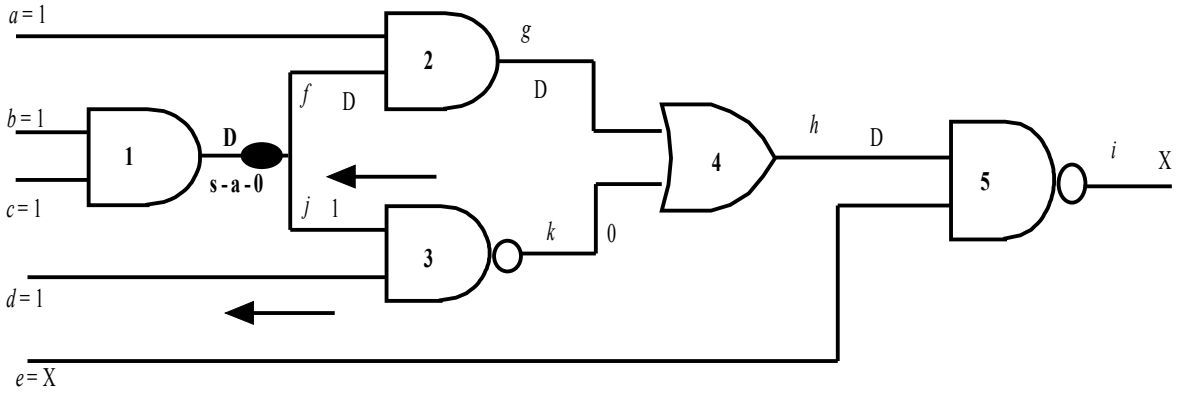


Figure 7(g): Backward implication

Assignment of net *j*=1 is inconsistency because it implies output of gate-1 to be 1, whereas it needs to be D (for fault sensitization). Inconsistency is shown in Figure 7(h)

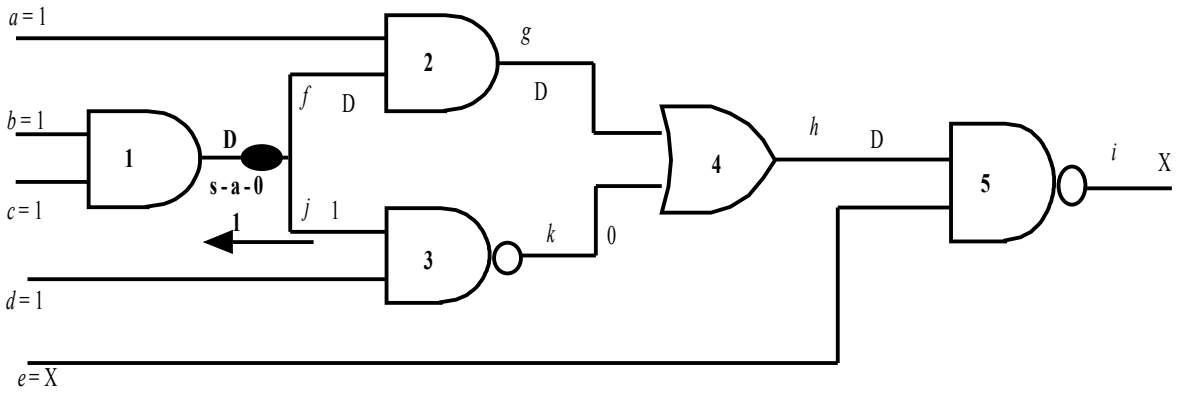
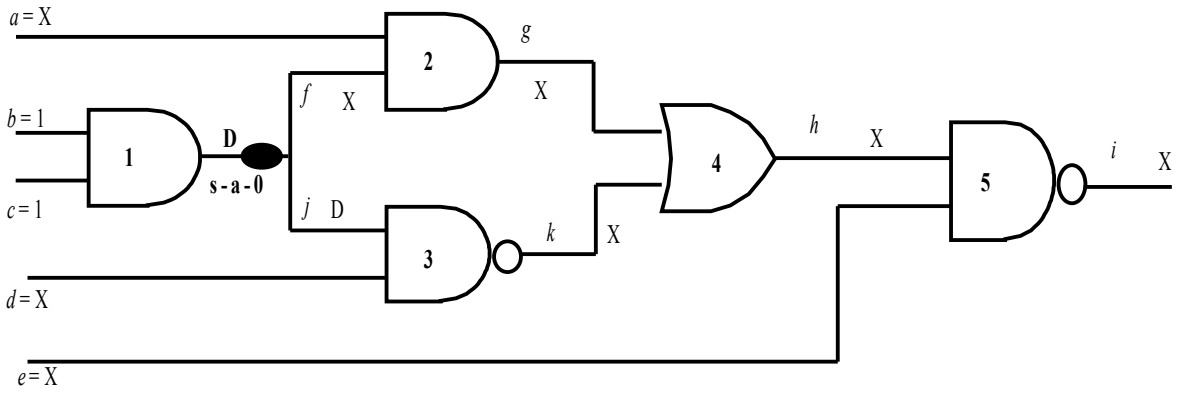


Figure 7(h): Inconsistency in net *j*

After this inconsistency we need to backtrack. To backtrack we need to find points were there were choices and we took one of them. In this example, there was a choice of selecting one gate (for propagating D) among two gates in the D-frontier; see Figure 7(d), where gate-2 was

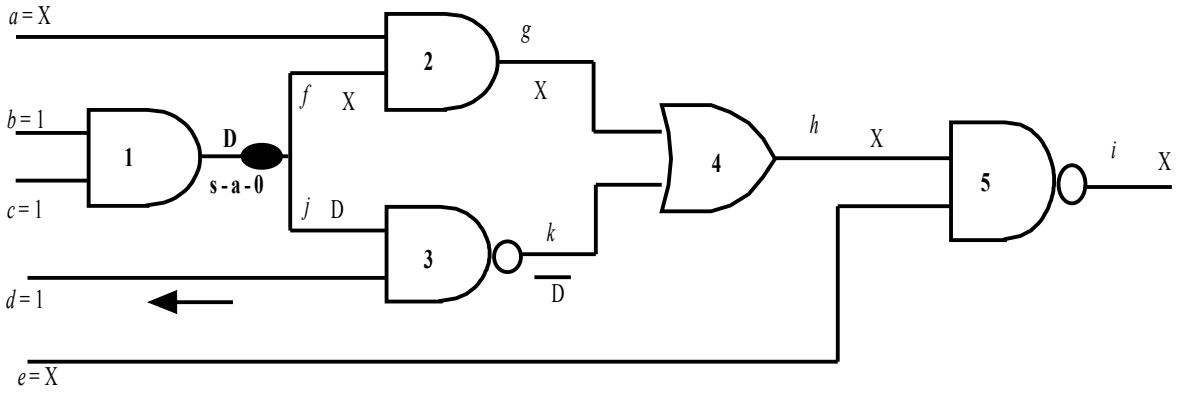
selected. Now let us backtrack by taking the other choice, i.e., consider gate-3 for D propagation; this is shown in Figure 7(i). It may be noted that for backtracking we have to undo all steps starting from where we got inconsistency to the point where another choice is to be tried. In Figure 7(i) all steps are undone (by making net values to X).



Select the other gate-3 from D-frontier ={2,3}
Modified D-frontier ={3}

Figure 7(i): Backtrack by considering gate-3 for D propagation

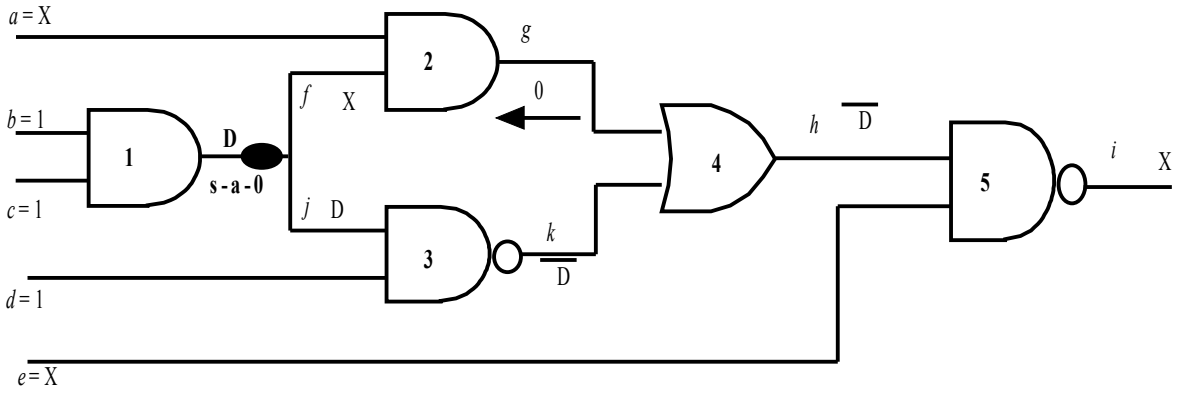
Similar to the step in Figure 7(e), we propagate D though gate-3 (the other choice). This makes net $k= \overline{D}$ by forward implication and d=1 by backward implication. The step is illustrated in Figure 7(j). After the propagation of $\overline{D}$ though gate-2, the new D-frontier comprises gate-4.



Propagate D through D-frontier and Backward Implication
Modified D-frontier ={4}

Figure 7(j): Propagation of D and implications

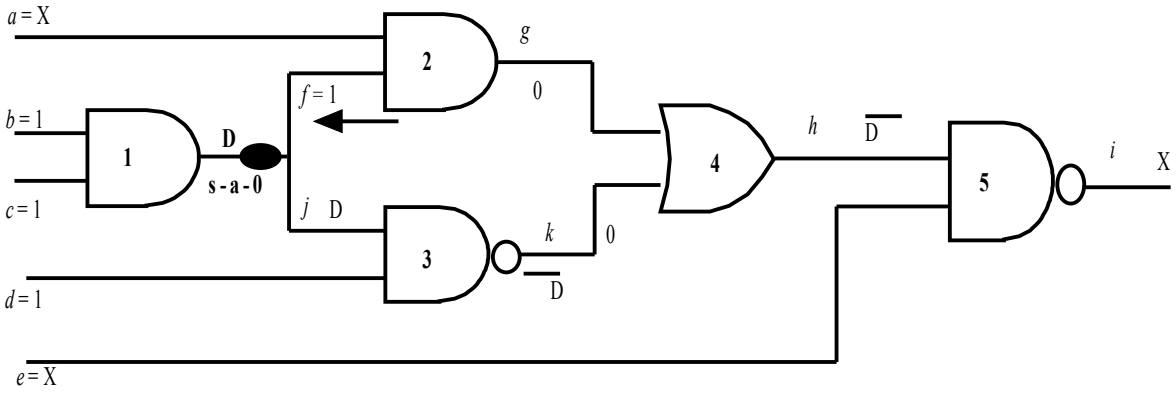
Now, $\overline{D}$ is propagated through gate-4, making D-frontier comprise gate-5. Propagation of $\overline{D}$ through gate-4 implies net *g*=0 by backward implication; also gate-2 now falls in J-frontier. These changes are illustrated in Figure 7(k).



Propagate D through D-frontier and Backward Implication
Modified D-frontier ={5}
J-frontier={2}

Figure 7(k): Propagation of $\overline{D}$, implications and D and J-frontier changes

In the next step, backward implication of the gate at J-frontier (gate-2) has two choices (i) *a*=X, *f*=0, (ii) *a*=0, *f*=X. Let us take the first choice. Assignment of net *f*=0 is inconsistency because it implies output of gate-1 to be 1, whereas it needs to be D (for fault sensitization). This is illustrated in Figure 7(l).



Backward Implication
Modified J-frontier={}
Inconsistency at net *f*

Figure 7(l): Inconsistency in net *f*

As discussed before, backtrack is required for this inconsistency. In this case, there was a choice of selecting values for input *a* and net *f*, by backward implication to set *g*=0; see Figure 7(l), where choice (i) was selected. Now let us backtrack by taking the other choice, i.e., *a*=0, *f*=X; this is shown in Figure 7(m). In Figure 7(m) all steps are undone (by making net values to X) required in this backtracking.

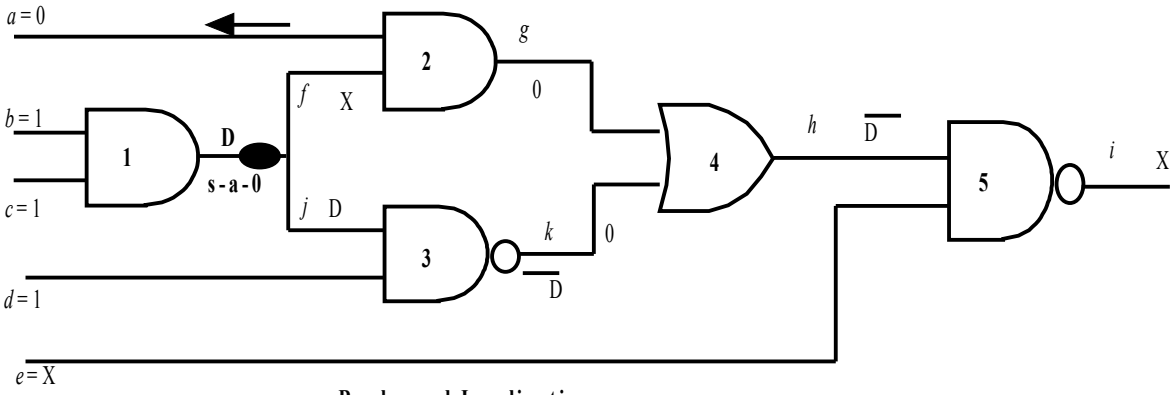


Figure 7(m): Backtrack by considering *a*=0, *f*=X for backward implication for *g*=0

Now, $\overline{D}$ is propagated through gate-5, which is in D-frontier. Now as gate-5 is driving a primary output, the fault effect is propagated successfully. This results in *e*=1, by backward implication. This finishes a successful generation of a test pattern by D-algorithm, as there is no inconsistency and fault effect ($\overline{D}$, here) reaches a primary output. Test pattern is *a*=0,*b*=1,*c*=1,*d*=1,*e*=1. This step is illustrated in Figure 7(n).

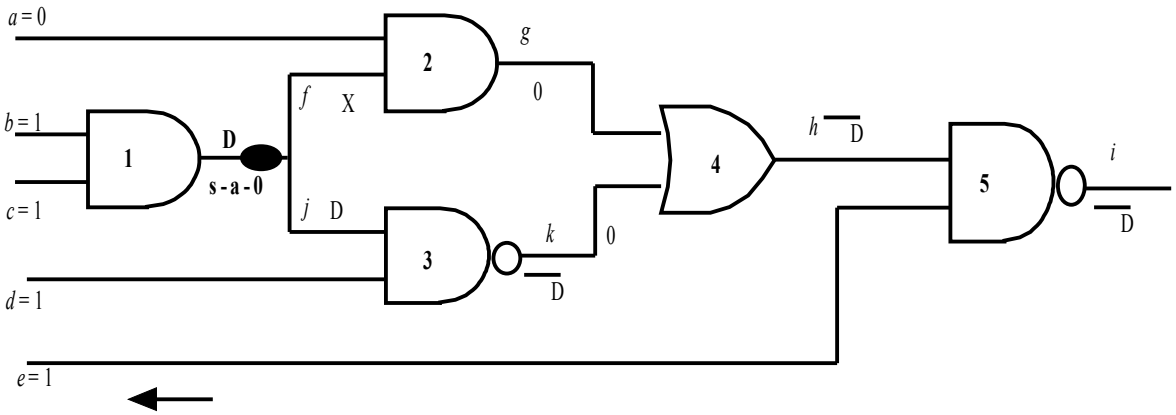


Figure 7(n): Backward implication for *e*=1 and successful test pattern generation

4. **Why advanced ATPG algorithms: PODEM [2] and FAN[3]**

In this course, we will not go into the details of the advanced ATPG algorithms. Interested readers can go through the references mentioned above. However, we will point out the drawbacks in the D-algorithm and briefly discuss how they were handled in the advanced ATPG algorithms.

If we observe the steps in the D-algorithm we can notice that it has three basic points

- Sensitize the fault location (i.e., D/$\overline{D}$ in the fault net)
- Propagation of D/$\overline{D}$ to a primary output, using *any* gates in D-frontier
- Use *any* values of signals as per forward and backward implications, to get the values of primary inputs, which sensitize the fault and propagate it to an output.

At any point, backtrack is required in case D-frontier is exhausted or an inconsistency is found.

So it may be stated that D-algorithm provides all the steps required for ATPG, however, says nothing about which of the alternatives are to be taken for propagation of D/$\overline{D}$ or values of signals as per forward and backward implications. In other words, the algorithm does not prioritize among the options available in case of propagation of D/$\overline{D}$ or signal assignment during implications. Also, in case of backtrack, information regarding the reason for inconsistency is not used for future selection of alternatives.

So the advanced algorithms basically improve D-algorithm by providing a guided choice of alternatives based on several testability measures namely,

- After a fault is sensitized, if there are more than one gate in the D-frontier, then choice can be made based on conditions like,
    - The shortest path (in terms of number of gates) from the fault site to a primary output is to be taken
    - The path having gate inputs (other than the one used for fault propagation) with low values of CC1 and CC0 (SCOAP values) are preferred than the ones with higher values.
- If more than one signal values are possible for nets after implication, consider the one that will result in low CC1 and CC0 values.
- Use the reasons for inconsistency for future selection of alternatives

5. **Conclusion**

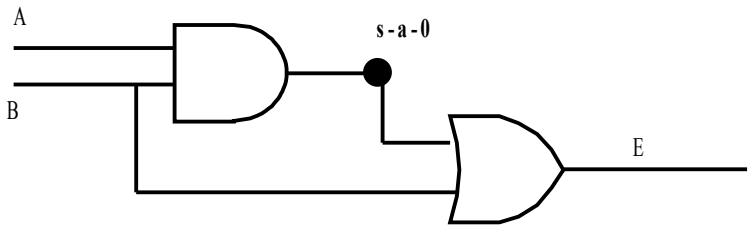
   Till this point in the course you might be surprised because we have talked only about testing of combinational circuits. However, most of the practical circuits are sequential ones. In the next module we will introduce ATPG for sequential circuits. Following that we will discuss a design for testability (DFT) technique called scan chain which converts sequential circuits to virtual combinational ones. So all the techniques discussed for combinational circuits will hold for sequential ones with DFT.

References

1. J. P. Roth, "Diagnosis of Automata Failures: A Calculus and a Method," IBM Journal of Research and Development, vol. 10, no. 4, pp. 278–291, July 1966.
2. P. Goel, "An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits," IEEE Trans. on Computers, vol. C-30, no. 3, pp. 215–222, Mar. 1981.
3. H. Fujiwara, "FAN: A Fanout-Oriented Test Pattern Generation Algorithm," in Proc. of the International Symp. on Circuits and Systems, July 1985, pp. 671–674.

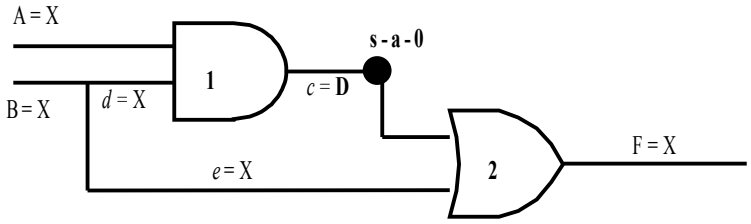
Questions and Answers

1. Using D-algorithm show that the circuit given below is un-testable:



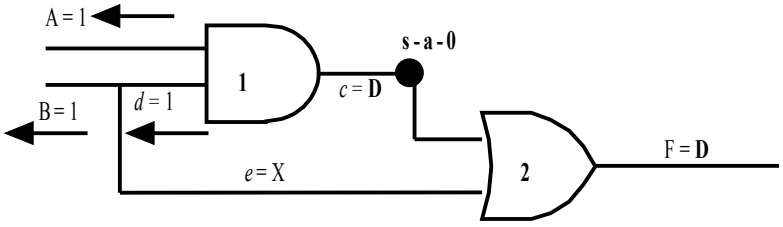
Answer:

In the first step, the signal values are initialized to X and the s-a-0 net is assigned to D. So the D-frontier is {2} and J-frontier is {1}.



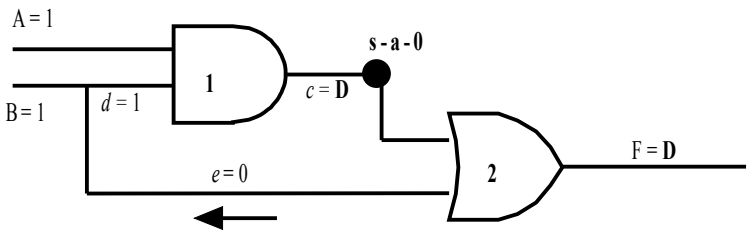
D-frontier ={2}
J-frontier ={1}

Now, as there is an unique D-drive, D is prorogated to gate-2, making signal F=D. By backward implications on gate-1, A=1,*d*=1 and B=1.



D-frontier ={}
Backward Implication

Also, by backward implication on gate-2, *e*=0. This is an inconsistency because, *e*=0 would make B=0, by backward implication.



Backward Implication
Inconstancy

It may be noted that this fault is not testable, because there is no alternative to backtrack, as there were no choices at any step.